SQL Server JSON Functions

HARSH KOTHADIYA

JSON

What is JSON ??

- JSON stands for JavaScript Object Notation.
- JSON is a lightweight data-interchange format that is easy for humans to read and simple for computers to parse.
- JSON is often used to exchange data between servers and web applications.
- JSON is built based on two main data structures: object and array.

Feature	Array	Array of Objects
Data type	Simple values	Objects
Structure	`[1,2,3]`	`[{},{}]`
Use case	List of items	Structured records
Example	`["A","B","C"]`	`[{"name":"A"},{"name":"B"}]`

Data Type	JSON Structure
One Row	Object `{}`
Multiple Rows	Array of Objects `[{} , {}]`

ISJSON()

ISJSON()

Check JSON is valid or not

Valid Json return 1

In-Valid Json return 0

EX-1

```sql
DECLARE @JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":121
}'

SELECT ISJSON(@JSON_DATA) AS 'ANS';
```

0% No issues found

Results | Messages

ANS
1

EX-2

```sql
DECLARE @JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":121
}'

SELECT ISJSON(@JSON_DATA) AS 'ANS';
```

0% No issues found

Results | Messages

ANS
0

JSON_VALUE()

JSON_VALUE()

- JSON_VALUE() always returns NVARCHAR(4000).
- JSON_VALUE() AUTO MATIC DATA TYPE CONVERT TO DATA STORING TABLE.
- BELOW #ONE_ROW_JSON_DATA THIS TABLE COLUMN DATA TYPE ME CONVERT JSON_VALUE().
- Use TRY_CAST so no error one row insert see Ex-3

EX-1

```sql
DECLARE @ONE_ROW_JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":121
}'

DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.FIRST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.LAST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.DOB'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.ROLE'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.HIGHEST_T20I_RUNS');


SELECT * FROM #ONE_ROW_JSON_DATA;
```

@ONE_ROW_JSON_DATA
INSIDE ALL KEY DATA TYPE
MATCH WITH TABLE REQUIRED DATA TYPE SO
SAME DATA STORE

Results | Messages

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	121

EX-2

```sql
DECLARE @ONE_ROW_JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"ABC"
}'

DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.FIRST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.LAST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.DOB'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.ROLE'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.HIGHEST_T20I_RUNS');


SELECT * FROM #ONE_ROW_JSON_DATA;
```

@ONE_ROW_JSON_DATA INSIDE HIGHEST_T20I_RUNS IN JSON STRING PASS BUT
TABLE INT REQUIRED JSON_VALUE AUTO MATIC CONVERT DATA TYPE BUT ONLY IF SAME DATA TYPE SO AUTO MATIC CONVERT OTHER WISE ERROR...

```
Msg 245, Level 16, State 1, Line 19
Conversion failed when converting the nvarchar value 'ABC' to data type int.
```

EX-3

```sql
DECLARE @ONE_ROW_JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"ABC"
}'

DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.FIRST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.LAST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.DOB'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.ROLE'),
    TRY_CAST(JSON_VALUE(@ONE_ROW_JSON_DATA,'$.HIGHEST_T20I_RUNS') AS INT);

SELECT * FROM #ONE_ROW_JSON_DATA;
```

EX-2 SOLUTION USE TRY_CAST
SO THAT COLUMN NULL VALUE STORE...

WRONG DATA TYPE VALUE PASS

MEANS THAT COLUMN NULL VALUE STORE....

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	NULL

EX-4

```sql
DECLARE @ONE_ROW_JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"121"
}'

DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.FIRST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.LAST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.DOB'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.ROLE'),
    TRY_CAST(JSON_VALUE(@ONE_ROW_JSON_DATA,'$.HIGHEST_T20I_RUNS') AS INT);
SELECT * FROM #ONE_ROW_JSON_DATA;
```

Results | Messages

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	121

EX-5

```sql
DECLARE @ONE_ROW_JSON_DATA VARCHAR(MAX) = '{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"121"
}'

DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.FIRST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.LAST_NAME'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.DOB'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.ROLE'),
    JSON_VALUE(@ONE_ROW_JSON_DATA,'$.HIGHEST_T20I_RUNS');
SELECT * FROM #ONE_ROW_JSON_DATA;
```

@ONE_ROW_JSON_DATA INSIDE 121 PASS AS STRING BUT 121 NUMBER SO JSON AUTO MATIC CONVERT NOT REQUIRED TRY_CAST

Results | Messages

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	121

MORE THAN ONE ROW INSERT

USE OPENJSON

Multiple row insert with out with

Use TRY_CAST

so, no any error see example-3

EX-3 SYNTAX USING NO ANY ERROR

```sql
--EX-1
DECLARE @MORE_THAN_ONE_ROW_JSON_DATA VARCHAR(MAX) =
'[
{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":121
}
,
{
    "FIRST_NAME":"Virat",
    "LAST_NAME":"Kohli",
    "DOB":"1988-11-05",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":122
}
]'
```

100% SURE INSIDE
CORRECT DATA TYPE
PASS ONLY
THIS CASE EX-1
TYPE OF SYNTAX
USING

```sql
DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(VALUE,'$.FIRST_NAME'),
    JSON_VALUE(VALUE,'$.LAST_NAME'),
    JSON_VALUE(VALUE,'$.DOB'),
    JSON_VALUE(VALUE,'$.ROLE'),
    JSON_VALUE(VALUE,'$.HIGHEST_T20I_RUNS')
FROM OPENJSON(@MORE_THAN_ONE_ROW_JSON_DATA)

SELECT * FROM #ONE_ROW_JSON_DATA;
```

Results | Messages

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	121
2	Virat	Kohli	1988-11-05	Batsman	122

```sql
--EX-2
DECLARE @MORE_THAN_ONE_ROW_JSON_DATA VARCHAR(MAX) =
'[
{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"ABC"
}
,
{
    "FIRST_NAME":"Virat",
    "LAST_NAME":"Kohli",
    "DOB":"1988-11-05",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":122
}
]'
```

```sql
DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(VALUE,'$.FIRST_NAME'),
    JSON_VALUE(VALUE,'$.LAST_NAME'),
    JSON_VALUE(VALUE,'$.DOB'),
    JSON_VALUE(VALUE,'$.ROLE'),
    JSON_VALUE(VALUE,'$.HIGHEST_T20I_RUNS')
FROM OPENJSON(@MORE_THAN_ONE_ROW_JSON_DATA)

SELECT * FROM #ONE_ROW_JSON_DATA;
```

Messages

```
Msg 245, Level 16, State 1, Line 31
Conversion failed when converting the nvarchar value 'ABC' to data type int.
```

```sql
--EX-3
DECLARE @MORE_THAN_ONE_ROW_JSON_DATA VARCHAR(MAX) =
'[
{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"ABC"
}
,
{
    "FIRST_NAME":"Virat",
    "LAST_NAME":"Kohli",
    "DOB":"1988-11-05",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":122
}
]'
```

```sql
DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

INSERT INTO #ONE_ROW_JSON_DATA
SELECT
    JSON_VALUE(VALUE,'$.FIRST_NAME'),
    JSON_VALUE(VALUE,'$.LAST_NAME'),
    JSON_VALUE(VALUE,'$.DOB'),
    JSON_VALUE(VALUE,'$.ROLE'),
    TRY_CAST(JSON_VALUE(VALUE,'$.HIGHEST_T20I_RUNS') AS INT)
FROM OPENJSON(@MORE_THAN_ONE_ROW_JSON_DATA)
```

Results | Messages

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	NULL
2	Virat	Kohli	1988-11-05	Batsman	122

Multiple row insert using with

1ST WITH RUN SO WITHOUT VARCHAR DATA TYPE CONSIDER
INSIDE WITH AS VARCHAR AND AFTR SELECT RUN
SO INSIDE SELECT USE TRY_CAST

SEE EXAMPLE-3

```sql
--EX-1
DECLARE @MORE_THAN_ONE_ROW_JSON_DATA VARCHAR(MAX) =
'[
{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":121
}
,
{
    "FIRST_NAME":"Virat",
    "LAST_NAME":"Kohli",
    "DOB":"1988-11-05",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":122
}
]'
```

100% SURE INSIDE
CORRECT DATA TYPE
PASS ONLY
THIS CASE EX-1
TYPE OF SYNTAX USING

```sql
DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);
```

```sql
INSERT INTO #ONE_ROW_JSON_DATA(
    FIRST_NAME,
    LAST_NAME,
    DOB,
    ROLE,
    HIGHEST_T20I_RUNS
)
SELECT
    FIRST_NAME,
    LAST_NAME,
    DOB,
    ROLE,
    HIGHEST_T20I_RUNS
FROM OPENJSON(@MORE_THAN_ONE_ROW_JSON_DATA)
WITH(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

SELECT * FROM #ONE_ROW_JSON_DATA;
```

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	121
2	Virat	Kohli	1988-11-05	Batsman	122

```sql
--EX-2
DECLARE @MORE_THAN_ONE_ROW_JSON_DATA VARCHAR(MAX) =
'[
{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"ABC"
}
,
{
    "FIRST_NAME":"Virat",
    "LAST_NAME":"Kohli",
    "DOB":"1988-11-05",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":122
}
]'

DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);
```

```sql
INSERT INTO #ONE_ROW_JSON_DATA(
    FIRST_NAME,
    LAST_NAME,
    DOB,
    ROLE,
    HIGHEST_T20I_RUNS
)
SELECT
    FIRST_NAME,
    LAST_NAME,
    DOB,
    ROLE,
    HIGHEST_T20I_RUNS
FROM OPENJSON(@MORE_THAN_ONE_ROW_JSON_DATA)
WITH(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);

SELECT * FROM #ONE_ROW_JSON_DATA;
```

```
Msg 245, Level 16, State 1, Line 31
Conversion failed when converting the nvarchar value 'ABC' to data type int.
```

```sql
--EX-3
DECLARE @MORE_THAN_ONE_ROW_JSON_DATA VARCHAR(MAX) =
'[
{
    "FIRST_NAME":"Rohit",
    "LAST_NAME":"Sharma",
    "DOB":"1987-04-30",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":"ABC"
}
,
{
    "FIRST_NAME":"Virat",
    "LAST_NAME":"Kohli",
    "DOB":"VK",
    "ROLE":"Batsman",
    "HIGHEST_T20I_RUNS":122
}
]'
```

```sql
DROP TABLE IF EXISTS #ONE_ROW_JSON_DATA;

CREATE TABLE #ONE_ROW_JSON_DATA(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB DATE,
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS INT
);
```

```sql
INSERT INTO #ONE_ROW_JSON_DATA(
    FIRST_NAME,
    LAST_NAME,
    DOB,
    ROLE,
    HIGHEST_T20I_RUNS
)
SELECT
    FIRST_NAME,
    LAST_NAME,
    TRY_CAST(DOB AS DATE),
    ROLE,
    TRY_CAST(HIGHEST_T20I_RUNS AS INT)
FROM OPENJSON(@MORE_THAN_ONE_ROW_JSON_DATA)
WITH(
    FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20),
    DOB VARCHAR(10),
    ROLE VARCHAR(50),
    HIGHEST_T20I_RUNS VARCHAR(5)
);

SELECT * FROM #ONE_ROW_JSON_DATA;
```

	FIRST_NAME	LAST_NAME	DOB	ROLE	HIGHEST_T20I_RUNS
1	Rohit	Sharma	1987-04-30	Batsman	NULL
2	Virat	Kohli	NULL	Batsman	122

JSON_OBJECT()

🔥 1. `JSON_OBJECT()`

👉 Creates a **single JSON object (key–value pairs)**

✅ Syntax

```sql
JSON_OBJECT(key1, value1, key2, value2, ...)
```

✅ Example

```sql
SELECT JSON_OBJECT(
    'FIRST NAME', FIRST_NAME,
    'LAST NAME', LAST_NAME
)
FROM CRICKETER_DETAIL;
```

🔥 Output (Row-wise)

```json
{"FIRST NAME":"Rohit","LAST NAME":"Sharma"}
{"FIRST NAME":"Virat","LAST NAME":"Kohli"}
```

💡 Use Case

- Convert **one row → one JSON object**
- API-style key-value structure

```sql
USE CRICKETDB
--SELECT * FROM CRICKETER_DETAIL;

SELECT
JSON_OBJECT(
    'FIRST NAME':FIRST_NAME,
    'LAST NAME':LAST_NAME
)AS 'FIRST_AND_LAST_NAME_JSON_OBJECT'

FROM CRICKETER_DETAIL;
```

Key Insight

- In `JSON_OBJECT(key, value)` → **keys are just strings**, so:
  - `'FIRST NAME'` ✅ valid
  - `'FIRST_NAME'` ✅ valid

Results | Messages

	FIRST_AND_LAST_NAME_JSON_OBJECT
1	{"FIRST NAME":"Rohit","LAST NAME":"Sharma"}
2	{"FIRST NAME":"Virat","LAST NAME":"Kohli"}
3	{"FIRST NAME":"Sanju","LAST NAME":"Samson"}
4	{"FIRST NAME":"Jasprit","LAST NAME":"Bumrah"}
5	{"FIRST NAME":"Shivam","LAST NAME":"Dube"}


```
{
  "FIRST NAME": "Shivam",
  "LAST NAME": "Dube"
}
```

JSON_ARRAY()

🚀 2. `JSON_ARRAY()`

👉 Creates a **JSON array (list)**

✅ Syntax

```sql
JSON_ARRAY(value1, value2, ...)
```

✅ Example

```sql
SELECT JSON_ARRAY('Rohit', 'Virat', 'Sanju');
```

🔥 **Output**

```json
["Rohit", "Virat", "Sanju"]
```

```sql
SELECT JSON_ARRAY(JSON_OBJECT(...)) FROM table;
```

👉 You'll get **one array per row**, not a combined array.

```sql
USE CRICKETDB
--SELECT * FROM CRICKETER_DETAIL;

SELECT
JSON_ARRAY(
    JSON_OBJECT(
        'FIRST NAME':FIRST_NAME,
        'LAST NAME':LAST_NAME
    )
)AS 'FIRST_AND_LAST_NAME_JSON_OBJECT_PER_ROW_CONVERT_JSON_ARRAY'

FROM CRICKETER_DETAIL;
```

FIRST_AND_LAST_NAME_JSON_OBJECT_PER_ROW_CONVERT_JSON_ARRAY
[{"FIRST NAME":"Rohit","LAST NAME":"Sharma"}]
[{"FIRST NAME":"Virat","LAST NAME":"Kohli"}]
[{"FIRST NAME":"Sanju","LAST NAME":"Samson"}]
[{"FIRST NAME":"Jasprit","LAST NAME":"Bumrah"}]
[{"FIRST NAME":"Shivam","LAST NAME":"Dube"}]


```
[
  {
    "FIRST NAME": "Shivam",
    "LAST NAME": "Dube"
  }
]
```

FOR JSON PATH

```sql
SELECT(
    SELECT
        FIRST_NAME AS 'First Name',
        LAST_NAME AS 'Last Name'
    FROM CRICKETER_DETAIL
    FOR JSON PATH
) AS 'ANS'
```

0% No issues found

Results | Messages

ANS
[{"First Name":"Rohit","Last Name":"Sharma"},{"...

ANS2.json | SQLQuery2.sq...\harsh (66))*


```json
[
  {
    "First Name": "Rohit",
    "Last Name": "Sharma"
  },
  {
    "First Name": "Virat",
    "Last Name": "Kohli"
  },
  {
    "First Name": "Sanju",
    "Last Name": "Samson"
  },
  {
    "First Name": "Jasprit",
    "Last Name": "Bumrah"
  },
  {
    "First Name": "Shivam",
    "Last Name": "Dube"
  }
]
```

FOR JSON PATH, WITHOUT_ARRAY_WRAPPER

```sql
SELECT(
    SELECT
        FIRST_NAME AS 'First Name',
        LAST_NAME AS 'Last Name'
    FROM CRICKETER_DETAIL
    FOR JSON PATH, WITHOUT_ARRAY_WRAPPER
) AS 'ANS'
```

ANS
{"First Name":"Rohit","Last Name":"Sharma"},{"Fi...


```
{"First Name":"Rohit","Last Name":"Sharma"},
{"First Name":"Virat","Last Name":"Kohli"},
{"First Name":"Sanju","Last Name":"Samson"},
{"First Name":"Jasprit","Last Name":"Bumrah"},
{"First Name":"Shivam","Last Name":"Dube"}
```

🔥 Key Difference

Feature	FOR JSON PATH	WITHOUT_ARRAY_WRAPPER
Output Type	Array of objects	Single object
Brackets	`[ ]` present	Removed
Use Case	Multiple rows	Single row

JSON_QUERY()

❗ Important Reminder

- `JSON_QUERY()` works only with **object / array**

🔹 What is `JSON_QUERY()`?

`JSON_QUERY()` is used to **extract JSON objects or arrays** from a JSON string.

👉 It **returns JSON (not a scalar value)**.

🔹 Syntax

```sql
JSON_QUERY(expression, path)
```

- **expression** → JSON string (column or variable)
- **path** → JSON path (like `$.address`, `$.players`) ↓

🔹 Key Point (VERY IMPORTANT)

Function	Returns	Use Case
`JSON_VALUE()`	Single value	Name, age, etc.
`JSON_QUERY()`	Object / Array	Full JSON structure

```sql
SELECT
    FIRST_NAME,
    JSON_QUERY(
        (
            SELECT
                JSON_VALUE(STAT, '$.T20_50s') AS T20_50s,
                JSON_VALUE(STAT, '$.T20_100s') AS T20_100s
            FOR JSON PATH, WITHOUT_ARRAY_WRAPPER
        )
    ) AS T20_STATS
FROM CRICKETER_DETAIL;
```

Results | Messages

	FIRST_NAME	T20_STATS
1	Rohit	{"T20_50s":"29","T20_100s":"4"}
2	Virat	{"T20_50s":"37","T20_100s":"1"}
3	Sanju	{"T20_50s":"22","T20_100s":"3"}
4	Jasprit	{"T20_50s":"0","T20_100s":"0"}
5	Shivam	{"T20_50s":"5","T20_100s":"0"}


```
{
    "T20_50s": "29",
    "T20_100s": "4"
}
```